HOTELPRO: HOTEL MANAGEMENT SOLUTION

Project Description:

- **HotelPro** is an end-to-end enterprise hotel management and reservation web application built using the modern MERN stack (MongoDB, Express.js, React.js, and Node.js) with secure JSON Web Token (JWT) role-based authentication and Bootstrap-powered luxury glassmorphic styling. The system streamlines hotel operations across independent boutique properties and large hospitality chains, providing real-time room inventory synchronization, interactive suite booking, automated 12% hospitality tax computations, multi-channel payment simulations with downloadable itemized invoices, automated room turnover dispatching to housekeeping staff upon guest checkout, and an executive analytics suite tracking live occupancy rates, Average Daily Rate (ADR), and comprehensive guest lifetime customer profiles.

Scenario:

In today's fast-paced hospitality industry, managing hotel operations efficiently and securely is more crucial than ever. At HotelPro, we offer a reliable and intuitive hotel management system designed to meet the evolving needs of modern hospitality. Whether you're handling reservations, managing guest information, or processing payments, our platform ensures seamless operations with advanced security and scalability. HotelPro provides a robust backend infrastructure that powers efficient booking management, real-time availability tracking, and secure user authentication. We prioritize data privacy, encryption, and compliance, ensuring that every guest interaction is protected from unauthorized access. With a user-friendly interface and seamless integrations, our system simplifies hotel management for both independent properties and large hospitality chains.

System Requirements:

To ensure smooth development, deployment, and usage of the HotelPro Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable application.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above):** Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 👉 Download Node.js (Installed: **v24.15.0**)
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries (Installed: **v11.12.1**).
- **React.js:** A JavaScript library for building dynamic and responsive user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.

- **Express.js:** A lightweight web framework for building RESTful APIs.
- **MongoDB:** A NoSQL database used to store structured and unstructured data related to **Users, Rooms, Bookings, and Housekeeping Tasks** 👉 Download MongoDB (Installed & Active Service on Port **27017**).
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub:** For version control and collaborative development.

2. Hardware Requirements

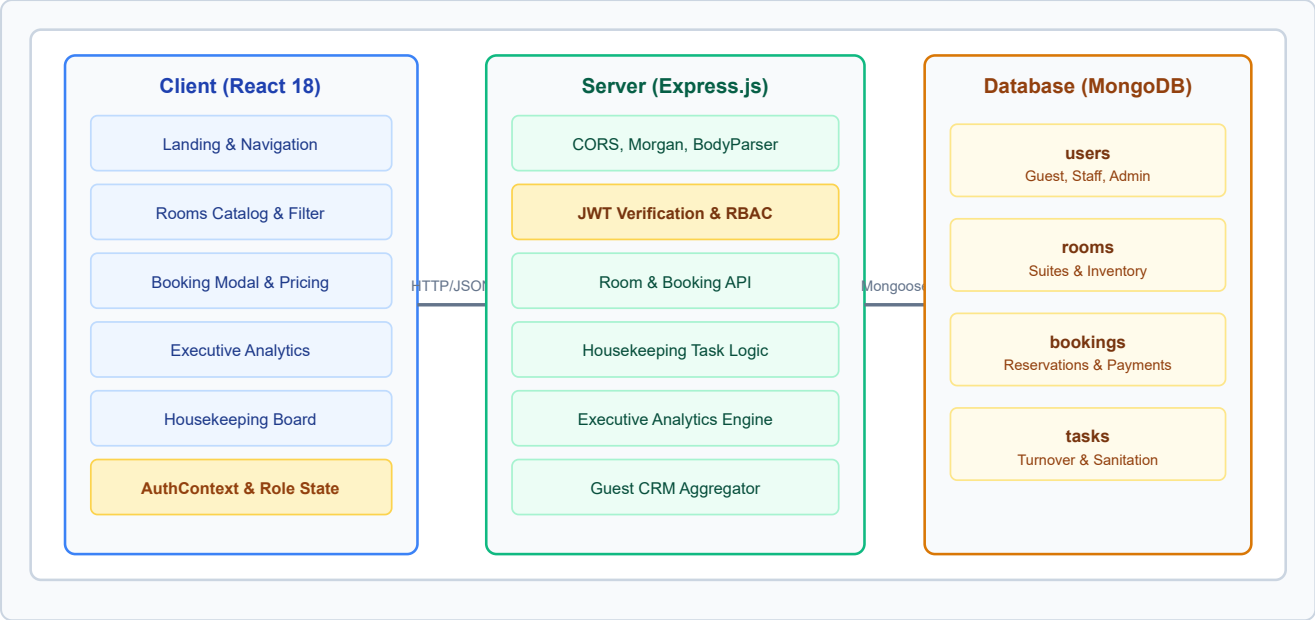
Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display:** 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

Epic 1 :- Project Architecture: 1hr

Project architecture defines the structured design of components, their interactions, and technologies used to build and deliver the application. Let's dive deep into this project's architecture.

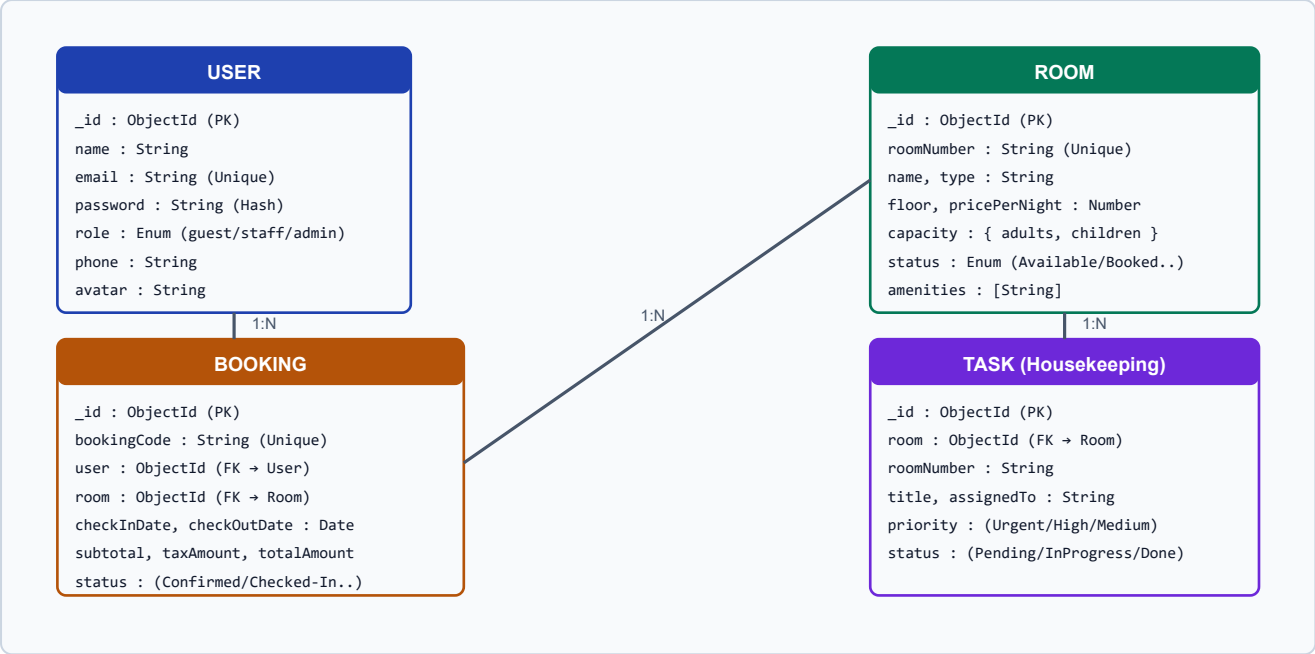
- **Technical Architecture: (Sample)**



○ In-detail explanation

The technical architecture coordinates three distinct layers: the **Client Layer** (React 18 with Vite build tooling, Bootstrap 5 UI, and Context API for global session state); the **Application Gateway & Logic Layer** (Node.js & Express.js REST API incorporating CORS, Morgan logger, JSON Web Token authorization middleware, and modular controllers); and the **Database Layer** (MongoDB on port 27017 utilizing Mongoose schemas to persist `users`, `rooms`, `bookings`, and `tasks`).

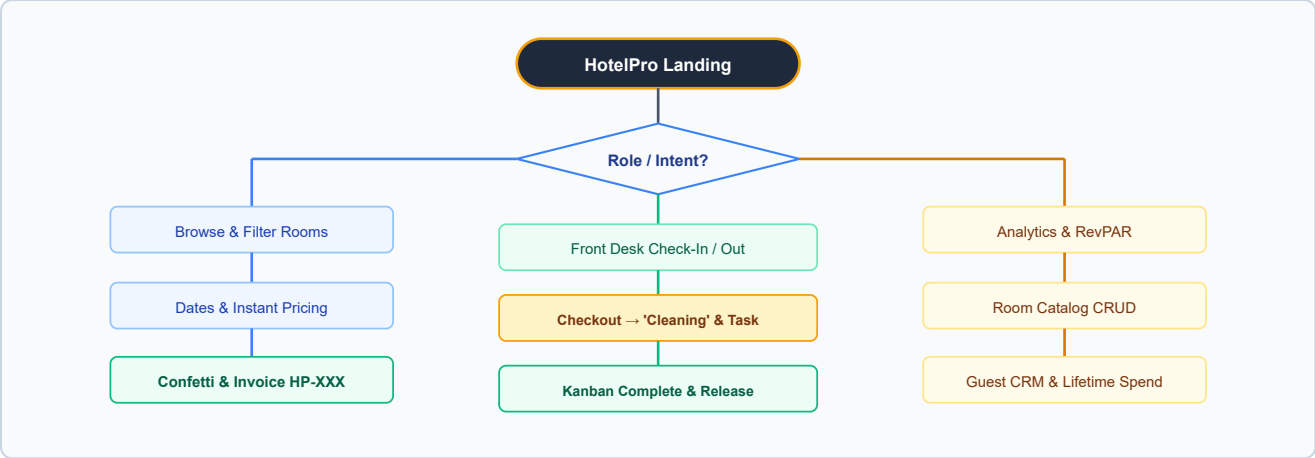
- **ER Diagram:(Sample)**



○ In-detail explanation

The data schema defines 4 primary MongoDB entities: **USER** stores account credentials and role tiers (`guest` , `staff` , `admin`) with hashed passwords; **ROOM** maintains inventory attributes (roomNumber, category, pricePerNight, capacity, and live status: `Available` , `Booked` , `Cleaning` , `Maintenance`); **BOOKING** links guests and rooms with stay dates, subtotal, 12% hospitality tax, payment method, and reservation lifecycle; and **TASK** coordinates room sanitation queues linked directly to Room ObjectIds.

● User Flow(Sample)



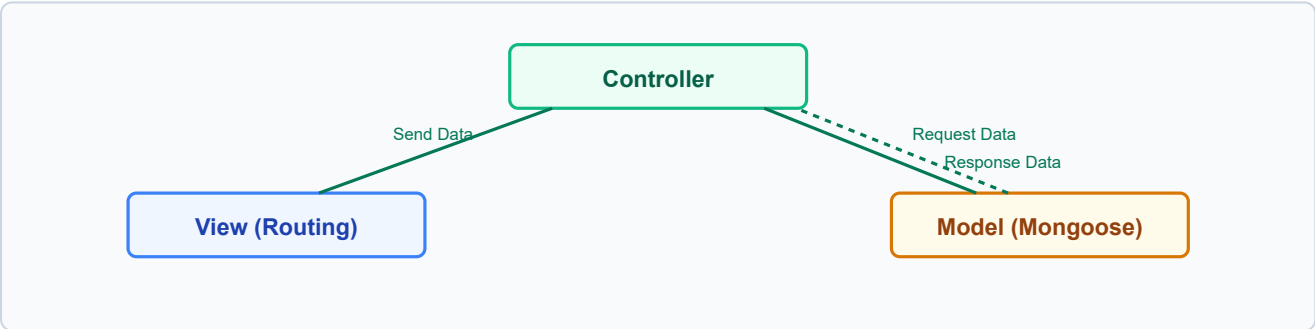
○ **In-detail explanation:** Highlights the end-to-end paths for Guests (room exploration, pricing, automated 12% tax, booking, receipt), Staff (check-in, check-out auto-turnover triggers, Kanban sanitation completion), and Admin (revenue metrics, floor plan matrix, and CRM).

● **Features:** JWT Authentication, Multi-category Room Tracking, Booking Engine with Tax Computation, Simulated Payment Processing, Housekeeping Kanban, Executive Dashboard, and Guest CRM.

● **Roles and responsibilities:**

- Guest : Books rooms, chooses payment methods, reviews past reservations, and downloads invoices.
- Staff : Manages Check-In/Check-Out, marks rooms for turnover, and updates task statuses.
- Admin : Manages room inventories, sets prices, analyzes occupancy/revenue, and audits all activities.

● **MVC Pattern**



MVC ARCHITECTURE PATTERN

The backend application is designed using the Model-View-Controller (MVC) architectural pattern, a widely adopted software design approach that promotes separation of concerns by organizing the application into three distinct layers: Model, Controller, and View (Routing Layer). This architecture improves code organization, maintainability, scalability, and collaboration among development teams.

Model Layer (Data Layer)

The Model Layer is responsible for managing the application's data and business entities. It defines the structure, relationships, validation rules, and constraints for the data stored in the database.

In this project, the Model layer is implemented using Mongoose, an Object Data Modeling (ODM) library for MongoDB. Mongoose provides schema-based data modeling, enabling developers to define data structures and perform database operations efficiently.

The responsibilities of the Model layer include:

- Defining database schemas and data models (`User.js` , `Room.js` , `Booking.js` , `Task.js`).
- Managing relationships between collections.
- Performing CRUD (Create, Read, Update, Delete) operations.
- Applying data validation and business rules.
- Interacting directly with the MongoDB database.

By centralizing data-related logic within the Model layer, the application ensures data consistency and simplifies database management.

Controller Layer (Business Logic Layer)

The Controller Layer acts as the bridge between incoming client requests and the application's data models. It contains the core business logic of the application and coordinates the flow of data between the routing layer and the database.

When a request is received, the controller:

1. Accepts and processes the request data.
2. Validates and transforms input where necessary.
3. Invokes appropriate Model methods to perform database operations.
4. Applies business rules and application logic.
5. Formats and sends the appropriate response back to the client.

This separation ensures that business logic remains independent of routing and database implementation details, resulting in cleaner and more maintainable code.

View Layer (Routing Layer)

In a backend REST API application, there is typically no traditional View layer responsible for rendering user interfaces. Instead, the Routing Layer serves a similar role by defining API endpoints, receiving HTTP requests, and directing them to the appropriate controller functions. While it is not a true View in the classical MVC sense, it acts as the entry point for client interactions with the backend application.

The routing layer is responsible for:

- Defining RESTful API endpoints.
- Mapping HTTP methods (GET, POST, PUT, PATCH, DELETE) to controller functions.
- Managing middleware execution such as authentication, authorization, validation, and logging.
- Forwarding requests to the appropriate controllers.

This layer serves as the entry point for all client interactions with the backend system and ensures proper request handling and endpoint organization.

Request Flow in MVC Architecture

The typical request lifecycle within the application follows these steps:

1. The client sends an HTTP request to a specific API endpoint.
2. The Route receives the request and forwards it to the corresponding Controller.
3. The Controller processes the request and invokes the necessary Model operations.

4. The Model interacts with the MongoDB database and returns the requested data.
5. The Controller formats the result and sends a response back through the Route.
6. The client receives the final response.

This structured workflow ensures clear responsibilities and efficient communication between application layers.

Epic 2:- Project Setup And Configuration. 30mins

• Creating project folder

1. Create a new folder with your `hotelpro`.
2. Inside that folder create two new folders.
3. Name one as **frontend** (Client).
4. Name another one as **backend** (Server).
5. Now open that folder in VS Code.

```
hotelpro/
├── backend/                  # Server folder (Node.js & Express API)
│   ├── config/db.js          # MongoDB Mongoose connection
│   ├── middleware/auth.js    # JWT verification & RBAC
│   ├── models/               # User, Room, Booking, Task schemas
│   ├── routes/               # Express route controllers
│   ├── .env                  # Environment variables
│   ├── package.json          # Backend dependencies
│   ├── seed.js               # Database seeding script
│   └── server.js             # Server entry point
├── frontend/                 # Client folder (React 18 + Vite)
│   ├── src/components/       # Navbar, RoomCard, BookingModal, InvoiceModal
│   ├── src/pages/            # LandingPage, RoomsPage, BookingsPage, etc.
│   ├── src/context/          # AuthContext with demo role switcher
│   ├── vite.config.js        # API proxy config to localhost:5000
│   └── package.json          # Frontend dependencies
├── .gitignore                # Ignoring node_modules and .env
├── package.json              # Concurrently orchestrator script
└── README.md                 # Project documentation
```

• Client setup (installing react app)

- Open the Client folder in the terminal of VS Code:

```
npm create vite@latest . -- --template react
```

- Select React framework from the given options:

```
Select a framework:
│ React
```

- Select JavaScript variant from the given options:

```
Select a variant:
| JavaScript
```

- Now lets navigate to the client folder by giving the following command:

```
cd frontend
```

- To install all the packages run the following command:

```
npm install bootstrap bootstrap-icons lucide-react react-router-dom canvas-confetti
```

- To start the React server type the following command:

```
npm run dev
```

- **Server setup (npm init)**

- Open Server folder in terminal of VS Code:

```
cd backend
npm init -y
```

- Create files: `server.js` , `seed.js` , `.env`
- Create folders: `models/` , `routes/` , `middleware/` , `config/`

Epic 3 :- Backend Development: 8hrs

- Reference video with voiceover or screen shots for backend code.

> **Note:- Proper Explanation needs to be there for each and every third-party-API consumed.**
> • **express**: HTTP server and RESTful routing engine.
> • **mongoose**: Strict schema definition, document validation, and ObjectId relationships.
> • **jsonwebtoken**: Issues cryptographically signed JWT bearer tokens for role authentication.
> • **bcryptjs**: One-way salt hashing (10 rounds) protecting user passwords.
> • **cors & morgan**: Cross-origin security and HTTP traffic debugging.

Epic 4 :- Database Development 2hrs

- Reference video with voiceover or screen shots for Database code.
- Schemas created: `User` (Admin, Staff, Guest), `Room` (Pricing, Status, Specs), `Booking` (Stay dates, 12% tax, Payments), `Task` (Turnover queues).

Epic 5:- Front-End Development. 8hrs

- Reference video with voiceover or screen shots for frontend code.

> **Note:- Proper Explanation needs to be there for each and every third-party-API consumed.**
> • **react-router-dom**: Declarative multi-page routing (Rooms, Bookings, Analytics, Housekeeping, CRM).
> • **bootstrap & bootstrap-icons**: Responsive layout, mobile toggles, status pills, and buttons.
> • **lucide-react**: Modern SVG icons for clean operational interfaces.
> • **canvas-confetti**: Celebratory confetti effect upon completed booking.

Epic 6:- Project Execution. 30mins

• Steps for Project execution

Step 1: Set Up the Frontend (React App):

- a) Open a terminal and navigate into the client folder:

```
cd frontend
```

- b) Once installation is done, start the React development server:

```
npm run dev
```

- c) The app should now be running on: **http://localhost:5173**

Step 2: Set Up the Backend (Express Server)

- a) Open a new terminal tab/window or split the terminal.
- b) Navigate into the server folder:

```
cd ../backend
```

Step 3: Configure Environment Variables

- a) Inside the server folder, create a new file named `.env` (no file extension).
- b) In that `.env` file, add your MongoDB connection string and JWT secret:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/hotelpro
JWT_SECRET=hotelpro_super_secure_jwt_secret_key_2026
```

Step 4: Start the Backend Server:

- Inside the same server folder, run the backend server using:

```
node server.js
```

- The server should start on: **http://localhost:5000**

• **Output ScreenShots with explanation:**

Screen / Module	Explanation of Outputs & Features
1. Landing Page	Hero section, category search widget, 6 Epics architectural breakdown, and live platform uptime indicators.
2. Rooms & Suites Catalog	Live room cards, floor specs, capacity, price per night, status badges (Available, Occupied, Cleaning), and category filters.
3. Booking & Payment Modal	Live stay calculation, 12% hospitality tax calculation, payment selection, room locking, and confetti celebration.
4. Official Invoice & Receipt	Printable hotel invoice with itemized room charges, taxes, transaction ID, and browser print trigger.
5. Housekeeping Kanban	Operational board with automated room turnover upon checkout; marking tasks complete auto-releases rooms to 'Available'.
6. Executive Dashboard	Live Occupancy %, Gross Revenue, Average Daily Rate (ADR), Room Category distribution, and real-time floor matrix.
7. Guest CRM Directory	Aggregated guest directory displaying lifetime stays, total expenditure, contact details, and stay history popovers.

• **Demo video link:** Local Demo at `http://localhost:5173` (Recorded in IDE browser session)

• **Code drive link / GitHub Repository:**

- 👉 **https://github.com/mahajanvedika9-lab/hotelpro**

> **Default Evaluation Accounts:**
> • **Admin:** `admin@hotelpro.com` / `admin123` (Passkey: `HOTELPRO_ADMIN_2026`)
> • **Staff:** `staff@hotelpro.com` / `staff123`
> • **Guest:** `guest@hotelpro.com` / `guest123`